

SQL Notes & Problem Solutions

Authored by: Arpit Raj, Lnmiit jaipur

1. Restaurant Expansion (Moving Average)

Table: Customer

Column Name	Type
customer_id	int
name	varchar
visited_on	date
amount	int

In SQL, (customer_id, visited_on) is the primary key for this table.

This table contains data about customer transactions in a restaurant.

visited_on is the date on which the customer with ID (customer_id) has visited the restaurant.

amount is the total paid by a customer.

You are the restaurant owner and you want to analyze a possible expansion (there will be at least one customer every day).

Compute the moving average of how much the customer paid in a seven days window (i.e., current day + 6 days before). average_amount should be rounded to two decimal places.

Return the result table ordered by visited_on in ascending order.

The result format is in the following example.

Example 1:

Input: Customer table:

customer_id	name	visited_on	amount
1	Jhon	2019-01-01	100
2	Daniel	2019-01-02	110
3	Jade	2019-01-03	120
4	Khaled	2019-01-04	130
5	Winston	2019-01-05	110
6	Elvis	2019-01-06	140
7	Anna	2019-01-07	150
8	Maria	2019-01-08	80
9	Jaze	2019-01-09	110
1	Jhon	2019-01-10	130
3	Jade	2019-01-10	150

Output:

visited_on	amount	average_amount
2019-01-07	860	122.86
2019-01-08	840	120
2019-01-09	840	120
2019-01-10	1000	142.86

Explanation:

1st moving average from 2019-01-01 to 2019-01-07 has an average_amount of $(100 + 110 + 120 + 130 + 110 + 140 + 150)/7 = 122.86$

2nd moving average from 2019-01-02 to 2019-01-08 has an average_amount of $(110 + 120 + 130 + 110 + 140 + 150 + 80)/7 = 120$

3rd moving average from 2019-01-03 to 2019-01-09 has an average_amount of $(120 + 130 + 110 + 140 + 150 + 80 + 110)/7 = 120$

4th moving average from 2019-01-04 to 2019-01-10 has an average_amount of $(130 + 110 + 140 + 150 + 80 + 110 + 130 + 150)/7 = 142.86$

```

SELECT
    c1.visited_on,
    SUM(c2.amount) AS amount,
    ROUND(SUM(c2.amount) / 7, 2) AS average_amount
FROM
    (
        SELECT
            visited_on,
            SUM(amount) AS amount
        FROM Customer
        GROUP BY visited_on
    ) c1
JOIN
    (
        SELECT
            visited_on,
            SUM(amount) AS amount
        FROM Customer
        GROUP BY visited_on
    ) c2
ON c2.visited_on BETWEEN DATE_SUB(c1.visited_on, INTERVAL 6 DAY)
    AND c1.visited_on
GROUP BY c1.visited_on
HAVING COUNT(*) = 7
ORDER BY c1.visited_on;

```

Why *HAVING COUNT(*) = 7*?

After aggregating, each row represents one day.

So for a given visited_on:

If there are exactly 7 rows in the joined range, then all 7 consecutive days exist.

Otherwise, it's not a complete 7-day window.

2. Daily Sales (Unique Leads & Partners)

Table: DailySales

Column Name	Type
date_id	date
make_name	varchar
lead_id	int
partner_id	int

There is no primary key (column with unique values) for this table. It may contain duplicates. This table contains the date and the name of the product sold and the IDs of the lead and partner it was sold to.

The name consists of only lowercase English letters.

For each date_id and make_name, find the number of distinct lead_id's and distinct partner_id's.

Return the result table in any order.

The result format is in the following example.

Example 1:

Input: DailySales table:

date_id	make_name	lead_id	partner_id
2020-12-8	toyota	0	1
2020-12-8	toyota	1	0
2020-12-8	toyota	1	2
2020-12-7	toyota	0	2
2020-12-7	toyota	0	1
2020-12-8	honda	1	2
2020-12-8	honda	2	1
2020-12-7	honda	0	1
2020-12-7	honda	1	2
2020-12-7	honda	2	1

Output:

date_id	make_name	unique_leads	unique_partners
2020-12-8	toyota	2	3
2020-12-7	toyota	1	2
2020-12-8	honda	2	2
2020-12-7	honda	3	2

```
SELECT date_id,
make_name,
COUNT(DISTINCT lead_id) AS unique_leads,
COUNT(DISTINCT partner_id) AS unique_partners

FROM DailySales
GROUP BY date_id,make_name;
```

3. Employees Total Time

Table: Employees

Column Name	Type
emp_id	int
event_day	date
in_time	int
out_time	int

(emp_id, event_day, in_time) is the primary key (combinations of columns with unique values) of this table.

The table shows the employees' entries and exits in an office.

event_day is the day at which this event happened, in_time is the minute at which the employee entered the office, and out_time is the minute at which they left the office.

in_time and out_time are between 1 and 1440.

It is guaranteed that no two events on the same day intersect in time, and $\text{in_time} < \text{out_time}$.

Write a solution to calculate the total time in minutes spent by each employee on each day at the office. Note that within one day, an employee can enter and leave more than once. The time spent in the office for a single entry is $\text{out_time} - \text{in_time}$.

Return the result table in any order.

Input: Employees table:

emp_id	event_day	in_time	out_time
1	2020-11-28	4	32
1	2020-11-28	55	200
1	2020-12-03	1	42
2	2020-11-28	3	33
2	2020-12-09	47	74

Output:

day	emp_id	total_time
2020-11-28	1	173
2020-11-28	2	30
2020-12-03	1	41
2020-12-09	2	27

Employee 1 has three events: two on day 2020-11-28 with a total of $(32 - 4) + (200 - 55) = 173$, and one on day 2020-12-03 with a total of $(42 - 1) = 41$.

```
SELECT event_day as day,  
emp_id,  
SUM(out_time-in_time) AS total_time  
  
FROM Employees  
GROUP BY emp_id,event_day;
```

4. Ad-Free Sessions

Table: Playback

Column	Type
session_id	int
customer_id	int
start_time	datetime
end_time	datetime

Table: Ads

Column	Type
ad_id	int
customer_id	int
timestamp	datetime

A session is ad-free if no ad was shown during that session.
Return the IDs of all ad-free sessions.

```
SELECT session_id
FROM Playback p
WHERE NOT EXISTS (
    SELECT 1
    FROM Ads a
    WHERE a.customer_id = p.customer_id
        AND a.timestamp BETWEEN p.start_time AND p.end_time
);
```


5. Matches Statistics

Table: Teams

Column	Type
team_id	int
team_name	varchar

Table: Matches

Column	Type
home_team_id	int
away_team_id	int
home_team_goals	int
away_team_goals	int

Return for every team:

team_name, matches_played, points, goal_for, goal_against, goal_diff

Order by: points DESC, goal_diff DESC, team_name ASC

```

SELECT
    t.team_name,
    COUNT(*) AS matches_played,
    SUM(points) AS points,
    SUM(goal_for) AS goal_for,
    SUM(goal_against) AS goal_against,
    SUM(goal_for - goal_against) AS goal_diff
FROM Teams t
JOIN (
    SELECT
        home_team_id AS team_id,
        CASE
            WHEN home_team_goals > away_team_goals THEN 3
            WHEN home_team_goals = away_team_goals THEN 1
            ELSE 0
        END AS points,
        home_team_goals AS goal_for,
        away_team_goals AS goal_against
    FROM Matches

    UNION ALL

    SELECT
        away_team_id,
        CASE
            WHEN away_team_goals > home_team_goals THEN 3
            WHEN away_team_goals = home_team_goals THEN 1
            ELSE 0
        END,
        away_team_goals,
        home_team_goals
    FROM Matches
) m
ON t.team_id = m.team_id
GROUP BY t.team_id, t.team_name
ORDER BY
    points DESC,
    goal_diff DESC,
    team_name;

```

Original Matches table

home	away	home_goals	away_goals
A	B	2	1
C	A	1	1

Notice something:

Each row contains statistics for two teams.

But in the final answer, we want statistics per team.

So we first split every match into two records.

```
-- Step 1 – Home team's perspective
SELECT
  home_team_id AS team_id,
  CASE
    WHEN home_team_goals > away_team_goals THEN 3
    WHEN home_team_goals = away_team_goals THEN 1
    ELSE 0
  END AS points,
  home_team_goals AS goal_for,
  away_team_goals AS goal_against
FROM Matches
```

6. Confirmation Rate

Table: Signups

Column Name	Type
user_id	int
time_stamp	datetime

user_id is the column of unique values for this table.

Each row contains information about the signup time for the user with ID user_id.

Each row of this table indicates that the user with ID user_id requested a confirmation message at time_stamp and that confirmation message was either confirmed ('confirmed') or expired without confirming ('timeout').

The confirmation rate of a user is the number of 'confirmed' messages divided by the total number of requested confirmation messages. The confirmation rate of a user that did not request any confirmation messages is 0. Round the confirmation rate to two decimal places.

Write a solution to find the confirmation rate of each user. Return the result table in any order.

Table: Confirmations

Column Name	Type
user_id	int
time_stamp	datetime
action	ENUM

(user_id, time_stamp) is the primary key.

user_id is a foreign key to Signups table.

action is ENUM ('confirmed', 'timeout')

Example 1:

Input: Signups table:

user_id	time_stamp
3	2020-03-21 10:16:13
7	2020-01-04 13:57:59
2	2020-07-29 23:09:44
6	2020-12-09 10:39:37

Confirmations table:

user_id	time_stamp	action
3	2021-01-06 03:30:46	timeout
3	2021-07-14 14:00:00	timeout
7	2021-06-12 11:57:29	confirmed
7	2021-06-13 12:58:28	confirmed
7	2021-06-14 13:59:27	confirmed
2	2021-01-22 00:00:00	confirmed
2	2021-02-28 23:59:59	timeout

Output:

user_id	confirmation_rate
6	0.00
3	0.00
7	1.00
2	0.50

Explanation:

User 6 did not request any confirmation messages. The confirmation rate is 0.

User 3 made 2 requests and both timed out. The confirmation rate is 0.

User 7 made 3 requests and all were confirmed. The confirmation rate is 1.

User 2 made 2 requests where one was confirmed and the other timed out. The confirmation rate is $1 / 2 = 0.5$.

```

SELECT

s.user_id,
ROUND(
    CASE WHEN COUNT(c.action)=0 THEN 0
    ELSE
    SUM(
        CASE WHEN c.action='timeout' THEN 0
        ELSE 1
        END
    )/COUNT(c.action)
    END,
    2
)

AS confirmation_rate
FROM Signups s
LEFT JOIN Confirmations c
ON s.user_id=c.user_id
GROUP BY s.user_id;

```

7. Understanding SQL Joins

Match on `left.customer_id = right.customer_id`

Left table

customer_id	customer
1	Ava
2	Bo
4	Dee

Right table

customer_id	level
1	Gold
3	Silver
4	Bronze

INNER JOIN · Result

customer_id	customer	level
1	Ava	Gold
4	Dee	Bronze

LEFT JOIN · Result

customer_id	customer	level
1	Ava	Gold
2	Bo	NULL
4	Dee	Bronze

RIGHT JOIN · Result

customer_id	customer	level
1	Ava	Gold
3	NULL	Silver
4	Dee	Bronze

FULL OUTER JOIN · Result

Keep every row; missing customers or levels become NULL

customer_id	customer	level
1	Ava	Gold
2	Bo	NULL
3	NULL	Silver
4	Dee	Bronze

8. Drop Type 1 Orders for Customers With Type 0 Orders

Table: Orders

Column	Type
order_id	int
customer_id	int
order_type	int

order_type = 0 → Type 0 order

order_type = 1 → Type 1 order

Rule

If a customer has at least one Type 0 order, then remove all their Type 1 orders.
Otherwise, keep all their orders.

Example

Input:

order_id	customer_id	order_type
1	1	0
2	1	1
3	2	1
4	2	1
5	3	0

Customer 1 (Has 0, 1): Has a Type 0 → Remove Type 1. Keep 0.

Customer 2 (Has 1, 1): No Type 0. Keep both.

Customer 3 (Has 0): Keep it.

Final Output:

order_id
1
3
4
5

```
SELECT *
FROM Orders o1
WHERE order_type = 0
OR NOT EXISTS (
    SELECT 1
    FROM Orders o2
    WHERE o2.customer_id = o1.customer_id
        AND o2.order_type = 0
);
```

Explanation

Keep the row if:

- It is already a Type 0 order, or*
- The customer does not have any Type 0 order.*

```
SELECT o1.*  
FROM Orders o1  
LEFT JOIN Orders o2  
ON o1.customer_id = o2.customer_id  
AND o2.order_type = 0  
WHERE o1.order_type = 0  
OR o2.customer_id IS NULL;
```

Detailed Step-by-Step Explanation

Step 1

```
SELECT o1.*
```

We want to return the original orders.

So we're selecting rows from o1.

Step 2

```
FROM Orders o1
```

This is our main table.

Suppose o1 is:

order_id	customer_id	order_type
1	1	0
2	1	1
3	2	1
4	2	1
5	3	0

Step 3

```
LEFT JOIN Orders o2
```

We join the same table again.

Why?

Because for every order we want to ask "Does this customer have a Type 0 order?"

We cannot answer that by looking at the current row alone.

So we compare the row with the same table.

Step 4

```
ON o1.customer_id = o2.customer_id
```

Now every order is matched with all orders of the same customer.

Example: Customer 1 has order types {0, 1}.

The join temporarily becomes:

o1.order	o1.type	o2.order	o2.type
1	0	1	0
1	0	2	1
2	1	1	0
2	1	2	1

Lots of rows.

Step 5

AND o2.order_type=0

This is the important part.

Now we only keep Type 0 rows from o2.

Customer 1 becomes:

o1.order	o1.type	o2.order	o2.type
1	0	1	0
2	1	1	0

Notice: Every order now knows "Yes, this customer has a Type 0 order."

Customer 2 Original: orders {3, 4} (types {1, 1}).

There is no Type 0.

So after LEFT JOIN:

o1.order	o1.type	o2.order
3	1	NULL
4	1	NULL

This NULL is exactly what we need.

Why LEFT JOIN?

Suppose we used JOIN.

(Customer 2) There is no Type 0. So No matching row. Customer 2 completely disappears.

But we actually want to keep their orders.

So JOIN ❌, LEFT JOIN ✅

LEFT JOIN says: Even if there is no matching Type 0, keep the row, fill right table with NULL.

Step 6

Now apply:

WHERE o1.order_type=0 OR o2.customer_id IS NULL

First condition: o1.order_type=0

Keep every Type 0 order.

Customer 1: Keep order 1.

Customer 3: Keep order 5.

Second condition: o2.customer_id IS NULL

Remember NULL means: No Type 0 exists for this customer. That's Customer 2.

So Keep Order 3, Order 4.

Final answer

order 1, 3, 4, 5

Exactly correct.